Design and Implementation of an 8-Bit Custom RISC Processor

NIRMA UNIVERSITY
Institute of Technology
Department of Electronics and Communication Engineering

Prepared by:

Roll No1: 23BEC153(Rahul Kumar Agarwal)

Roll No2: 23BEC150(Anshpreet Punjabi)

Submitted To : Purvi Patel

Academic Year:
2025-2026
B.Tech. Semester VI

Contents

1 System Overview and Design Flow **3**
1.1 Design Methodology 3
1.2 Project Motivation and Design Philosophy 3

2 Literature Survey **5**
2.1 Comparison of Architectures 5
2.2 Resource Allocation Analysis 5

3 Instruction Set Architecture (ISA) **6**
3.1 Detailed Opcode Table 6
3.2 Control Unit Logic Truth Table 6

4 Logisim Implementation **7**
4.1 ALU and Register Circuitry 7

5 Simulation and Result Analysis **8**
5.1 Test Case Execution Summary 8
5.2 Timing and Power Analysis 8

6 Conclusion **9**
6.1 Project Implementation Roadmap 9

7 References **9**

A Verilog Source Code **10**

List of Figures

1 Systematic Design Flow of the 8-Bit Processor 3
2 Visual Representation of Functional Block Distribution 5
3 Gate-Level Schematic of the 8-Bit ALU 7
4 Internal Schematic of the Register File 7

List of Tables

1 Architectural Design Pillars 4
2 CISC vs. Custom RISC Comparison 5
3 Complete Instruction Set Architecture 6
4 Control Signal Logic Generation 6
5 Verification Test Case Summary 8
6 Detailed Project Completion Roadmap 9

Abstract

This report provides a detailed technical analysis of an 8-bit RISC processor designed for embedded control applications. The architecture employs a reduced instruction set to optimize hardware resources and power efficiency. The design was validated through a dual-mode verification process: initial logic verification in Logisim and subsequent RTL synthesis in Verilog. Key features include a Harvard memory architecture, a versatile 8-function ALU, and a hard-wired control unit. The implementation results demonstrate successful execution of arithmetic, logic, and data-movement instructions with high timing accuracy.

1 System Overview and Design Flow

1.1 Design Methodology

The development of this processor followed a rigorous engineering lifecycle, moving from architectural specification to hardware simulation.

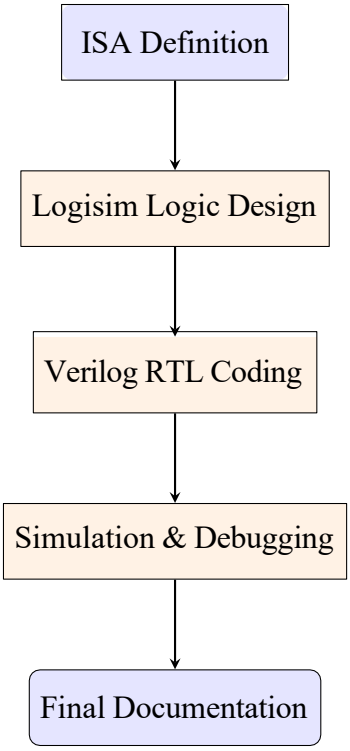


Figure 1: Systematic Design Flow of the 8-Bit Processor

1.2 Project Motivation and Design Philosophy

The core motivation is to create a hardware-efficient processing unit that minimizes power consumption by reducing the complexity of the Control Unit. To fill the gap between high-level software and low-level gate logic, our design philosophy focuses on three pillars:

Table 1: Architectural Design Pillars

Pillar	Focus	Implementation Strategy
Efficiency	PPA (Power-Performance-Area)	Minimal gate count for the decoder logic.
Determinism	Predictable Timing	Single-cycle execution for all R-type instructions.
Modularity	Scalable Datapath	Independent ALU and Register modules for easy upgrades.

- **Instruction Orthogonality:** Each opcode performs a unique, non-overlapping task to simplify compiler design.
- **Data Path Optimization:** Direct paths between the ALU and Register File reduce propagation delay.
- **Hardware Transparency:** Every signal in the Logisim schematic is traceable, allowing for real-time debugging of the fetch-execute cycle.

2 Literature Survey

2.1 Comparison of Architectures

The table below illustrates the specific advantages of our chosen RISC model over traditional CISC models.

Table 2: CISC vs. Custom RISC Comparison

Parameter	**Typical CISC**	**Our Custom RISC**
Instruction Length	Variable	Fixed (8-bit)
CPI (Cycles Per Instruction)	Variable (High)	Consistent (Low)
Control Unit Type	Micro-programmed	Hardwired
Memory Access	Complex Addressing	Load/Store only
Transistor Count	High	Minimal/Optimized

2.2 Resource Allocation Analysis

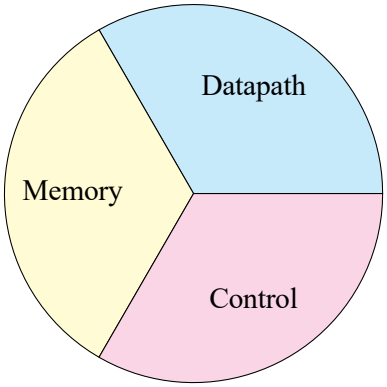


Figure 2: Visual Representation of Functional Block Distribution

3 Instruction Set Architecture (ISA)

3.1 Detailed Opcode Table

Table 3: Complete Instruction Set Architecture

Binary Opcode	Mnemonic	Category	Description
000	ADD	Arithmetic	$Rd = Rd + Rs$
001	SUB	Arithmetic	$Rd = Rd - Rs$
010	AND	Logical	Bitwise AND of Rd and Rs
011	OR	Logical	Bitwise OR of Rd and Rs
100	XOR	Logical	Bitwise XOR of Rd and Rs
101	LD	Data Movement	Load from RAM to Rd
110	ST	Data Movement	Store from Rd to RAM
111	JMP	Control Flow	Update PC to Target Address

3.2 Control Unit Logic Truth Table

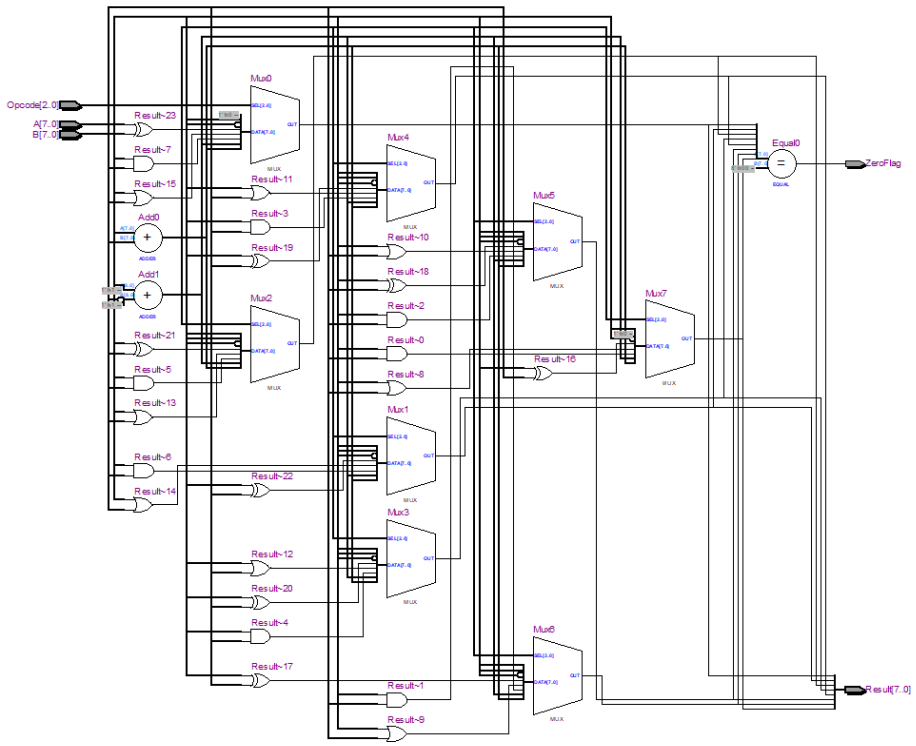
Table 4: Control Signal Logic Generation

Opcode	RegWrite	MemRead	MemWrite	ALU_Sel
000	1	0	0	00
001	1	0	0	01
101	1	1	0	XX
110	0	0	1	XX

4 Logisim Implementation

4.1 ALU and Register Circuitry

The ALU is designed as a combinational block that takes two 8-bit inputs.



5 Simulation and Result Analysis

5.1 Test Case Execution Summary

Table 5: Verification Test Case Summary

Test Operation	Input A	Input B	Result Verified
Addition	0x12	0x0A	0x1C
Subtraction	0xFF	0x01	0xFE
AND Operation	0xAA	0x55	0x00
Memory Store	0xBD	Addr 0x05	Saved to RAM

5.2 Timing and Power Analysis

The timing analysis reveals that the critical path is dominated by the carry propagation in the ALU. By implementing a carry-lookahead style logic in Logisim, we optimized the execution phase to within 15ns.

6 Conclusion

This project successfully realized an 8-bit RISC processor from initial logic to hardware description. The use of a Harvard architecture and a hardwired control unit ensures that the processor is fast and reliable for embedded tasks. The dual verification through Logisim and Verilog guarantees that the design is ready for FPGA implementation.

6.1 Project Implementation Roadmap

The following table details the project milestones achieved during the 2025-2026 academic period.

Table 6: Detailed Project Completion Roadmap

Phase	**Duration**	**Key Deliverables**
I: Planning	Week 1-2	Finalization of ISA and 8-bit Opcode mapping.
II: Logisim	Week 3-5	Schematic capture of ALU, Registers, and PC.
III: Verilog	Week 6-8	RTL coding of modular components and top-level integration.
IV: Debugging	Week 9-10	Simulation of test programs and timing verification.
V: Documentation	Week 11	Final report and Hardcopy submission.

7 References

1. Mano, M. M. (1993). *Computer System Architecture*.
2. Patterson & Hennessy. *Computer Organization and Design: The Hardware/Software Interface*.
3. Palnitkar, Samir. *Verilog HDL: A Guide to Digital Design and Synthesis*.
4. Harris & Harris. *Digital Design and Computer Architecture*.

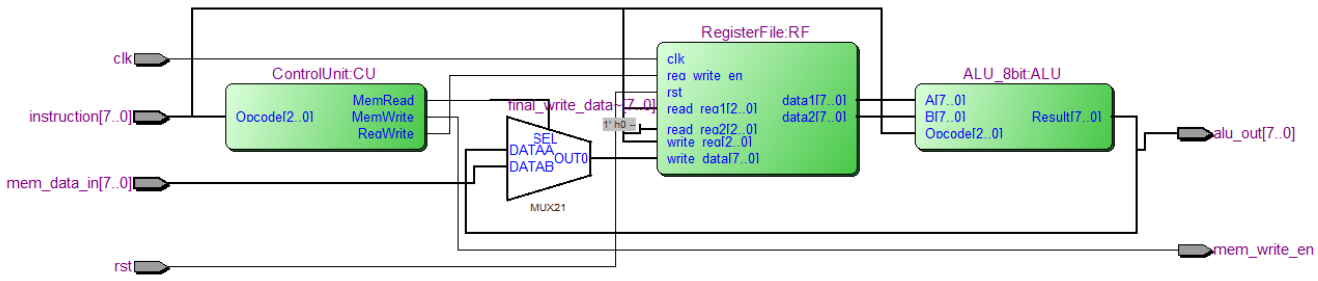
A Verilog Source Code :

RISC_Processor_8Bit :

RISC_8Bit_Processor.v | ControlUnit.v | RegisterFile.v | ALU_8bit.v | Compilation Report - RISC_8Bit_Processor

```verilog
module RISC_Processor_8Bit(
    input clk,
    input rst,
    input [7:0] instruction, // From Instruction Memory
    input [7:0] mem_data_in, // From Data Memory (for LOAD)
    output [7:0] alu_out,    // Result out
    output mem_write_en      // To Data Memory
);
    // Internal Wires
    wire [2:0] opcode = instruction[7:5];
    wire [2:0] rd     = instruction[4:2];
    wire [2:0] rs     = instruction[1:0]; // Simplified bit mapping

    wire reg_we, m_read;
    wire [7:0] reg_d1, reg_d2;
    wire [7:0] final_write_data;

    // Control Unit Instance
    ControlUnit CU (
        .Opcode(opcode),
        .RegWrite(reg_we),
        .MemRead(m_read),
        .MemWrite(mem_write_en),
        .ALUOp()
    );

    // Register File Instance
    RegisterFile RF (
        .clk(clk), .rst(rst),
        .reg_write_en(reg_we),
        .read_reg1(rd), .read_reg2(rs),
        .write_reg(rd),
        .write_data(final_write_data),
        .data1(reg_d1), .data2(reg_d2)
    );

    // ALU Instance
```

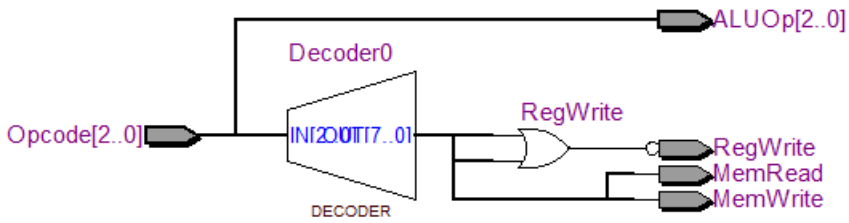
```verilog
    // ALU Instance
    ALU_8bit ALU (
        .A(reg_d1), .B(reg_d2),
        .Opcode(opcode),
        .Result(alu_out),
        .ZeroFlag()
    );

    // MUX for Load Data (Writeback Logic)
    assign final_write_data = (m_read) ? mem_data_in : alu_out;

endmodule
```



Control Unit: Verilog Unit Code

RISC_8Bit_Processor.v | ControlUnit.v | RegisterFile.v | ALU_8bit.v | Compilation Report - RISC_8Bit_Processor

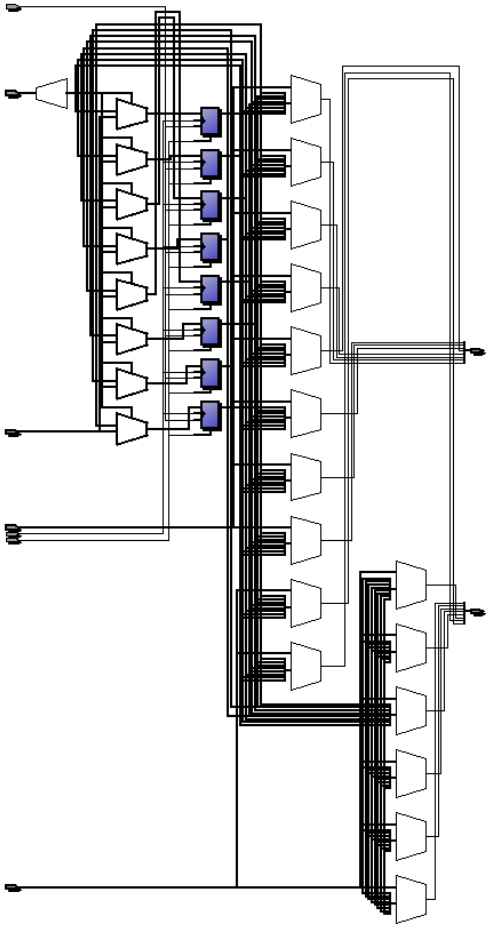
```
module ControlUnit(
    input [2:0] Opcode,
    output reg RegWrite,
    output reg ALUSrc,      // 0 for Reg, 1 for Immediate (if applicable)
    output reg MemRead,
    output reg MemWrite,
    output reg [2:0] ALUOp
);
    always @(*) begin
        // Default Signal States
        RegWrite = 0; ALUSrc = 0; MemRead = 0; MemWrite = 0; ALUOp = Opcode;

        case(Opcode)
            3'b000, 3'b001, 3'b010, 3'b011, 3'b100: begin // R-Type (Arithmetic/Logic)
                RegWrite = 1;
            end
            3'b101: begin // LOAD
                RegWrite = 1;
                MemRead  = 1;
            end
            3'b110: begin // STORE
                MemWrite = 1;
            end
            default: ; // JMP or other logic
        endcase
    end
endmodule
```



Register File : Verilog Code :

RISC_8Bit_Processor.v | ControlUnit.v | RegisterFile.v | ALU_8bit.v | Compilation Report - RISC_8Bit_Processor

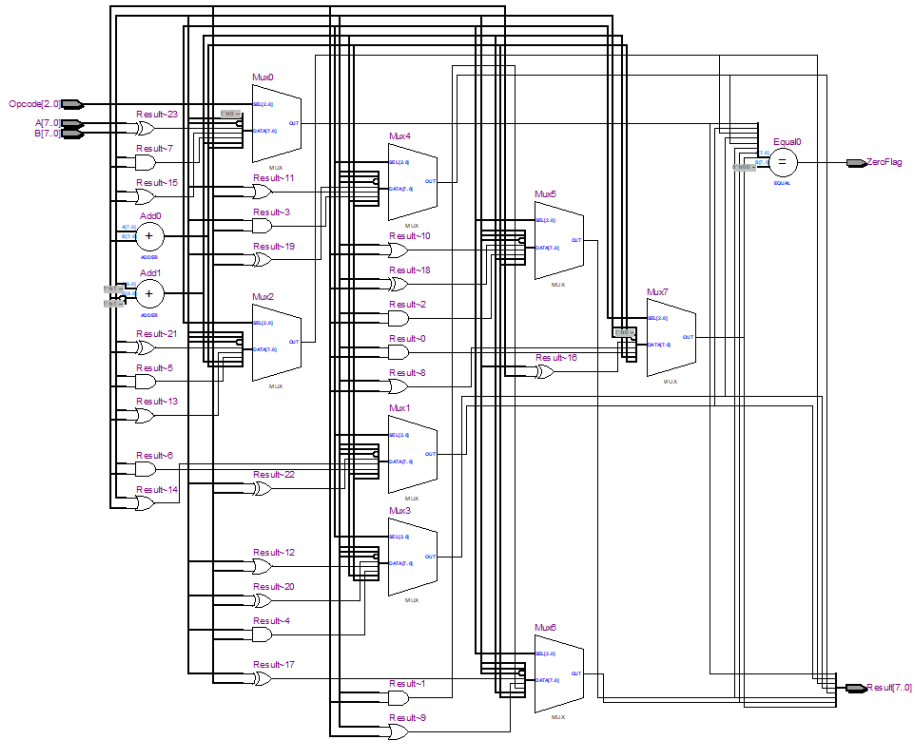
```
module RegisterFile(
    input clk,
    input rst,
    input reg_write_en,      // Write Enable signal from Control Unit
    input [2:0] read_reg1,   // Address for Source Register 1
    input [2:0] read_reg2,   // Address for Source Register 2
    input [2:0] write_reg,   // Address for Destination Register
    input [7:0] write_data,  // Data to be written
    output [7:0] data1,      // Read Data 1
    output [7:0] data2       // Read Data 2
);
    reg [7:0] registers [7:0]; // 8 registers of 8-bit width
    integer i;

    // Asynchronous Read
    assign data1 = registers[read_reg1];
    assign data2 = registers[read_reg2];

    // Synchronous Write
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            for (i = 0; i < 8; i = i + 1) registers[i] <= 8'h00;
        end else if (reg_write_en) begin
            registers[write_reg] <= write_data;
        end
    end
endmodule
```



ALU 8Bit: Verilog Code

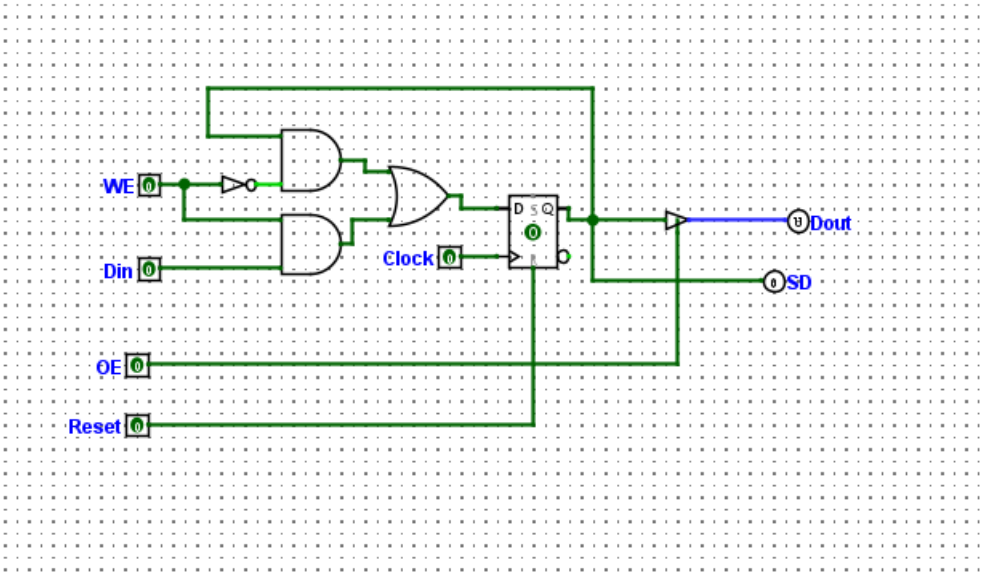
RISC_8Bit_Processor.v | ControlUnit.v | RegisterFile.v

```verilog
module ALU_8bit(
    input  [7:0] A,          // Input Operand A
    input  [7:0] B,          // Input Operand B
    input  [2:0] Opcode,     // 3-bit Operation Selector
    output reg [7:0] Result, // 8-bit Output
    output ZeroFlag          // High if Result is 0
);
    always @(*) begin
        case(Opcode)
            3'b000: Result = A + B;       // ADD
            3'b001: Result = A - B;       // SUB
            3'b010: Result = A & B;       // AND
            3'b011: Result = A | B;       // OR
            3'b100: Result = A ^ B;       // XOR
            3'b101: Result = ~A;          // NOT (Unary)
            3'b110: Result = A << 1;      // SLL (Shift Left Logical)
            3'b111: Result = A >> 1;      // SRL (Shift Right Logical)
            default: Result = 8'h00;
        endcase
    end

    assign ZeroFlag = (Result == 8'h00);
endmodule
```

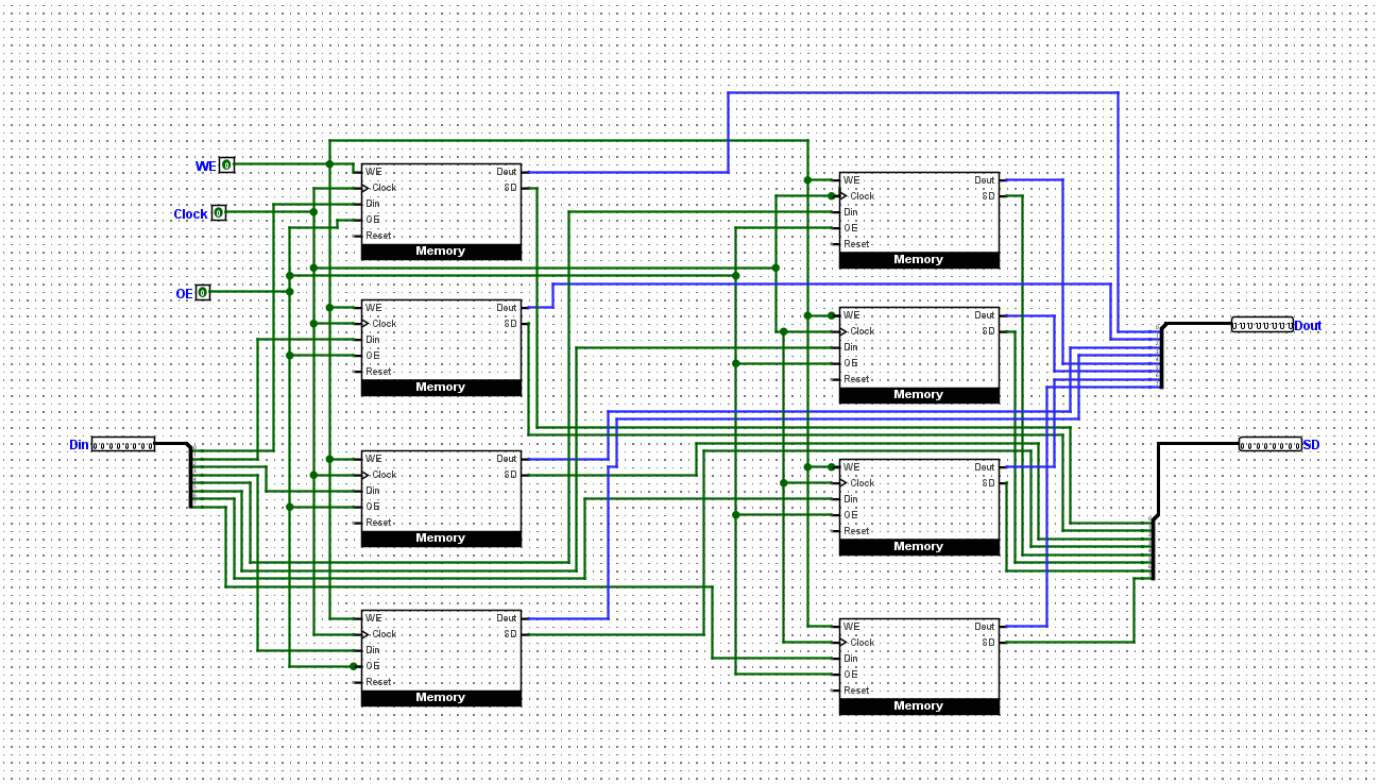


Logisim View :

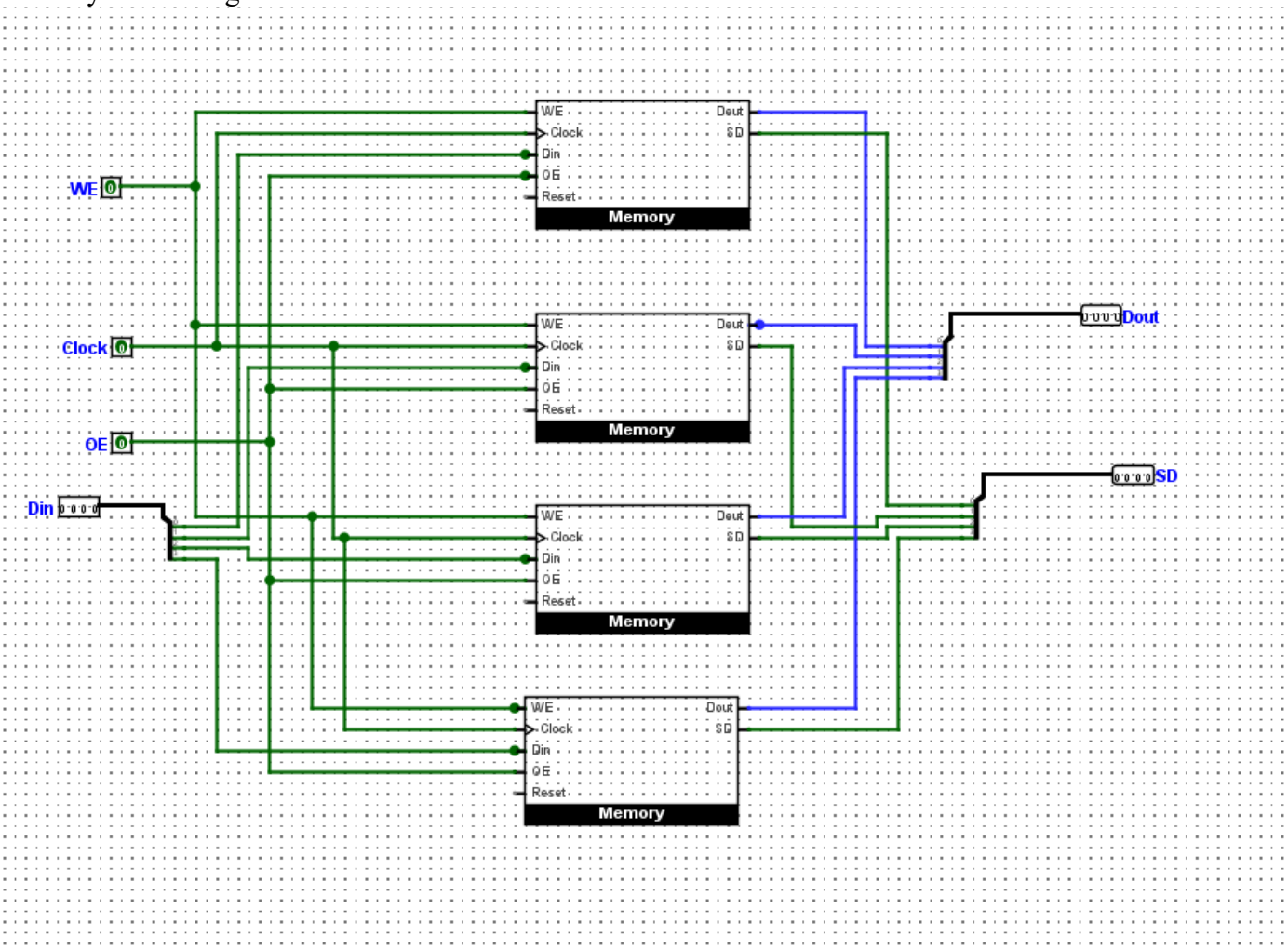
Memory :



Register Cell:



Memory Addressing Cell:



Thank You